EXPERIMENT 3

Write the Python program for the Water Jug Problem

Register Number: **192511013**

192511013
192511013
192511013

Aim

To write a Python program that measures exactly 2 litres of water using two jugs of capacity 4 litres and 3 litres (with no measuring marks), using Breadth First Search over the state space.

Algorithm

Step 1: Represent a state as (x, y), the amount of water in the 4 L jug and the 3 L jug. The start state is (0, 0).

Step 2: Create a queue with the start state and a parent table to remember how each state was reached.

Step 3: Remove the front state (x, y) from the queue. If x or y equals the target, trace back through the parent table to get the sequence of actions and stop.

Step 4: Generate the next states using the six rules: fill jug X, fill jug Y, empty jug X, empty jug Y, pour X into Y until Y is full or X is empty, and pour Y into X until X is full or Y is empty.

Step 5: For every new state that has not been seen before, record its parent and the action used, and add it to the queue.

Step 6: Repeat from step 3 until the target is reached or the queue becomes empty (no solution).

Step 7: Print the steps from (0, 0) to the goal state with the action performed at each step.

192511013

Program

```
from collections import deque

def water_jug(cap_x, cap_y, target):
    start = (0, 0)
    queue = deque([start])
    parent = {start: (None, "Start")}
    while queue:
        x, y = queue.popleft()
        if x == target or y == target:
            path = []
            state = (x, y)
            while state is not None:
                prev, action = parent[state]
                path.append((state, action))
                state = prev
            return path[::-1]
        pour_xy = min(x, cap_y - y)
        pour_yx = min(y, cap_x - x)
        moves = [
            ((cap_x, y), f"Fill jug X to {cap_x}"),
            ((x, cap_y), f"Fill jug Y to {cap_y}"),
            ((0, y), "Empty jug X"),
            ((x, 0), "Empty jug Y"),
            ((x - pour_xy, y + pour_xy), "Pour X into Y"),
            ((x + pour_yx, y - pour_yx), "Pour Y into X"),
        ]
        for state, action in moves:
            if state not in parent:
                parent[state] = ((x, y), action)
                queue.append(state)
    return None

if __name__ == "__main__":
    X, Y, T = 4, 3, 2
    print(f"Jug X = {X} L, Jug Y = {Y} L, Target = {T} L\n")
    result = water_jug(X, Y, T)
    if result:
        print(f"{'Step':<5}{'(X, Y)':<10}Action")
        print("-" * 40)
        for i, (state, action) in enumerate(result):
            print(f"{i:<5}{str(state):<10}{action}")
    else:
        print("Target cannot be measured")
```

192511013

Output

```
Jug X = 4 L, Jug Y = 3 L, Target = 2 L

Step (X, Y)    Action
----------------------------------------
0    (0, 0)    Start
1    (0, 3)    Fill jug Y to 3
2    (3, 0)    Pour Y into X
3    (3, 3)    Fill jug Y to 3
4    (4, 2)    Pour Y into X
```

192511013